



UNIVERSITÀ DI TRENTO

Relazione progetto Reti Neurali

Programmazione avanzata ed intelligenza artificiale

Studente: Daniele Pizzoli

Matricola: 243473

Corso: Programmazione avanzata ed
intelligenza artificiale
Docente: Prof. Paolo Rech

May 27, 2026

Contents

1	Introduzione	3
2	Scelta del task e criterio di valutazione	3
2.1	Perché una CNN per l'audio	3
3	Dataset e preprocessing	3
4	Architetture considerate	4
5	Addestramento e valutazione	4
6	Analisi delle prestazioni	5
6.1	Matrice di confusione del modello leggero	5
6.2	Matrice di confusione del modello ResNet	6
6.3	Analisi degli errori	6
6.4	Considerazioni sulla scelta del modello	7
7	Conclusioni	7

1 Introduzione

Il task scelto è la classificazione di comandi vocali a singola parola, un problema di classificazione multiclass su segnali audio. Il dataset adottato è Google SpeechCommands v0.02 e le classi considerate sono dieci: `yes`, `no`, `up`, `down`, `left`, `right`, `on`, `off`, `stop` e `go`.

Il flusso di lavoro segue un'impostazione tipica dei sistemi audio moderni: l'audio grezzo viene normalizzato, convertito in una rappresentazione tempo-frequenza e infine classificato da modelli neurali con complessità diversa. L'analisi non riguarda solo la precisione finale, ma anche l'effetto delle scelte progettuali sul comportamento del sistema, in particolare la struttura del modello e la qualità delle predizioni errate.

2 Scelta del task e criterio di valutazione

Il problema dei comandi vocali è adatto alla consegna perché consente di studiare un task reale, con un dataset ben definito e una separazione chiara tra training, validation e test set. Inoltre, la struttura del segnale audio rende naturale l'uso di reti convoluzionali, che possono lavorare sugli spettrogrammi come se fossero immagini.

La valutazione del sistema si basa principalmente su accuracy, matrice di confusione e analisi manuale dei campioni errati. Questa combinazione è utile perché l'accuracy da sola non spiega quali classi vengono confuse, mentre la confusione tra parole simili è spesso l'aspetto più interessante in un task audio di questo tipo. Per questo motivo i risultati vengono discussi non solo in termini numerici, ma anche in termini di robustezza e limiti del modello.

2.1 Perché una CNN per l'audio

La scelta di una CNN è naturale perché, una volta trasformato il segnale in spettrogramma Mel, il problema assume molte caratteristiche di una classificazione su immagini. I filtri convoluzionali possono individuare pattern locali nel tempo e nella frequenza, ad esempio bande di energia, transizioni rapide e strutture armoniche che caratterizzano la pronuncia di una parola.

Un altro vantaggio importante è la condivisione dei pesi: lo stesso kernel viene applicato su tutta la rappresentazione, quindi la rete riesce a cercare gli stessi indizi acustici in posizioni diverse dello spettrogramma. Il pooling riduce la sensibilità a piccole traslazioni e rende il modello più robusto rispetto a variazioni di pronuncia, velocità e allineamento temporale. Per questo motivo, per un task come questo, una CNN costituisce un baseline solido e coerente con il tipo di dato in ingresso.

3 Dataset e preprocessing

Il dataset utilizzato è SpeechCommands v0.02, presente nella cartella `dataset_audio/SpeechCommands/`. La pipeline implementata in `dataset.py` filtra il dataset originale per mantenere soltanto le dieci classi di interesse e suddivide i campioni nei tre sottoinsiemi standard: training, validation e testing.

Ogni segnale audio viene portato a una frequenza di campionamento di 16 kHz e normalizzato a una durata fissa di un secondo. Questa scelta semplifica il training e

rende l'input compatibile con il modello, che si aspetta tensori di dimensione costante. La conversione in spettrogramma Mel è gestita da `audio_utils.py`, dove l'audio viene trasformato con `MelSpectrogram` e convertito in dB tramite `AmplitudeToDB`. I parametri principali usati nella pipeline sono `N_FFT = 1024`, `HOP_LENGTH = 512` e `N_MELS = 64`.

In pratica, ogni clip viene prima adattata alla stessa lunghezza temporale e poi rappresentata come immagine monocanale, così da rendere applicabili architetture convoluzionali nate per la visione artificiale. Questo passaggio è centrale: per il problema considerato, lo spettrogramma conserva l'informazione utile della parola pronunciata molto meglio di una rappresentazione puramente numerica del segnale.

$$\text{spettrogramma_Mel} = \text{AmplitudeToDB}(\text{MelSpectrogram}(x))$$

4 Architetture considerate

Le architetture implementate nel progetto sono definite in `models.py`. Il modello principale usato per i test rapidi è **LightCNN**, una rete leggera con due blocchi convoluzionali e un classificatore fully connected finale. Questa soluzione è pensata per offrire un buon compromesso tra tempo di addestramento e capacità discriminativa, ed è una scelta coerente con un primo baseline richiesto dalla consegna.

Accanto al modello leggero è presente anche **AudioResNet**, ottenuto adattando una ResNet-18 all'ingresso monocanale degli spettrogrammi audio. La modifica principale consiste nella sostituzione del primo layer convoluzionale, che nella versione standard si aspetta tre canali RGB. Le architetture **AudioMobileNetV2** e **AudioAlexNet** sono state predisposte come alternative sperimentali, ma nel report restano secondarie rispetto al confronto tra LightCNN e ResNet, che è quello più utile per discutere il compromesso tra complessità e prestazioni.

La scelta di un modello più piccolo o più grande influisce direttamente sul comportamento della rete. LightCNN consuma meno risorse ed è più rapida da valutare, mentre ResNet tende a sfruttare meglio la capacità rappresentativa dei layer profondi, a costo di un'elaborazione più pesante. In entrambi i casi, l'uso di convoluzioni, normalizzazione e pooling risponde alla stessa esigenza: estrarre feature locali dal segnale audio e trasformarle in una rappresentazione progressivamente più astratta e più facile da classificare.

Il confronto tra i due modelli è coerente con la consegna del corso, perché permette di osservare come una rete più semplice possa essere sufficiente per un primo baseline, ma anche come una soluzione più profonda possa migliorare la separazione tra classi quando il problema diventa più ambiguo. In altre parole, la relazione non cerca un solo modello vincente, ma mostra il trade-off tra semplicità, costo computazionale e qualità della classificazione.

5 Addestramento e valutazione

L'addestramento è gestito da `train.py`, mentre la valutazione è eseguita da `evaluate.py`. Il processo di valutazione carica il checkpoint migliore salvato durante il training, esegue l'inferenza sul test set e produce due tipi di output: la matrice di confusione e un file CSV con i campioni errati. Quest'ultimo aspetto è particolarmente utile perché consente un'analisi qualitativa dei casi in cui il modello fallisce.

La funzione di loss usata nel problema è quella tipica della classificazione multi-classe, cioè la cross-entropy. In questo contesto il modello impara a massimizzare la probabilità della classe corretta a partire dallo spettrogramma in ingresso. Dal punto di vista sperimentale, la qualità finale non dipende solo dal numero di epoche o dal tasso di apprendimento, ma anche dalla stabilità della rappresentazione audio e dalla capacità della rete di distinguere comandi foneticamente simili.

6 Analisi delle prestazioni

6.1 Matrice di confusione del modello leggero

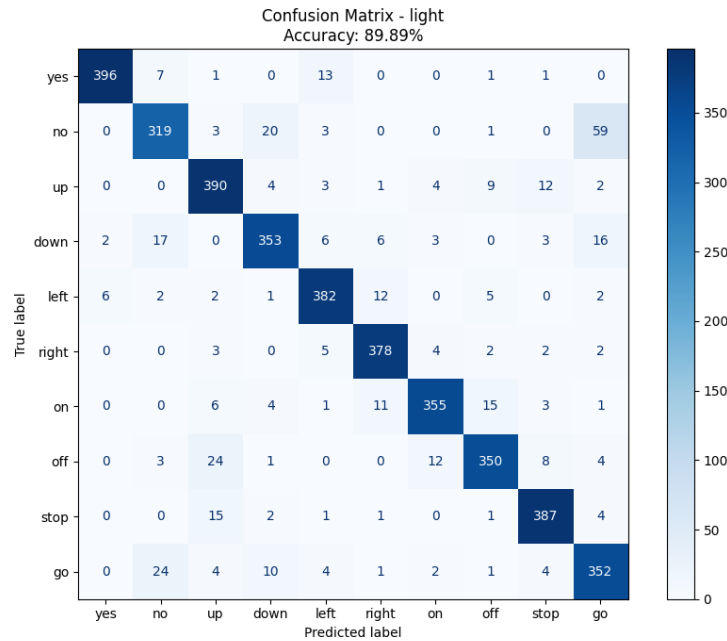


Figure 1: Matrice di confusione del modello LightCNN.

La matrice di confusione mostrata in Figura 1 evidenzia che il modello leggero riesce a riconoscere correttamente una parte consistente dei comandi, ma mostra ancora errori concentrati tra classi vicine dal punto di vista acustico. In particolare, i comandi brevi e molto simili tra loro tendono a essere scambiati con maggiore frequenza, segnale che la rete cattura bene le strutture più evidenti ma fatica nei casi ambigui.

6.2 Matrice di confusione del modello ResNet

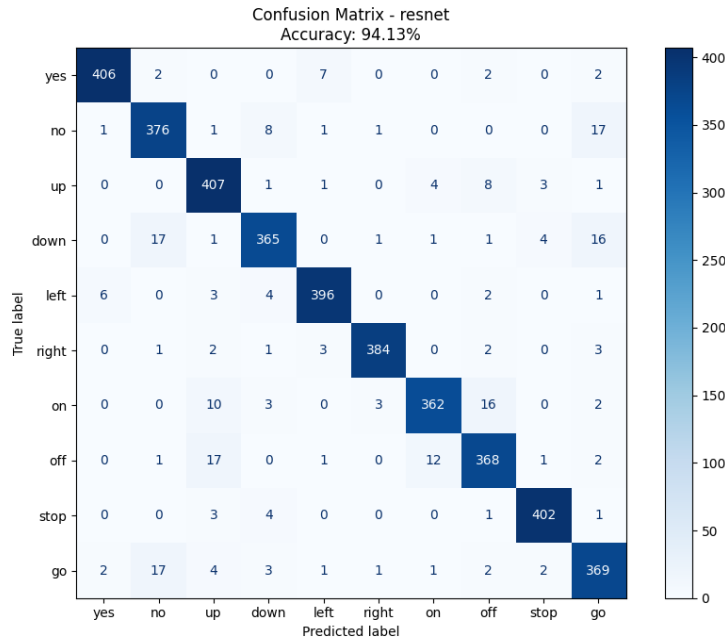


Figure 2: Matrice di confusione del modello AudioResNet.

Rispetto al modello leggero, la ResNet offre una rappresentazione più ricca e in genere produce una separazione più netta tra le classi. La Figura 2 mostra comunque che anche una rete più profonda non elimina del tutto le confusioni tra comandi simili, soprattutto quando la pronuncia è rapida o il segnale è meno netto. Il vantaggio principale della rete più profonda è una maggiore robustezza generale, mentre il limite rimane la naturale sovrapposizione acustica di alcune parole del dataset.

6.3 Analisi degli errori

Per capire meglio il comportamento del classificatore, i campioni errati vengono salvati nella cartella `eval_out/wrong_samples/`. I file CSV `wrong_light.csv` e `wrong_resnet.csv` permettono di ispezionare le predizioni sbagliate insieme alla confidenza associata alla classe predetta.

Questa analisi mette in evidenza un fenomeno ricorrente: molte delle errate classificazioni riguardano comandi foneticamente vicini oppure pronunciati in modo poco uniforme. Nei risultati osservati emergono errori frequenti tra **go**, **no**, **right**, **left**, **up**, **down**, **off** e **stop**. Il comportamento è coerente con la natura del dataset, in cui la variabilità tra parlanti, velocità di pronuncia e qualità della registrazione influisce molto sulla separabilità delle classi.

Questa parte della valutazione è importante anche dal punto di vista della consegna, perché mostra una lettura critica delle prestazioni: un modello può ottenere una buona accuracy complessiva e comunque fallire sistematicamente su alcune classi. Nel contesto del progetto questo aspetto conta più del valore assoluto dell'accuracy, perché dimostra la capacità di interpretare il modello e non soltanto di riportarne il punteggio.

6.4 Considerazioni sulla scelta del modello

Dal punto di vista pratico, LightCNN rappresenta una soluzione adatta quando si desidera un sistema semplice, veloce e facilmente addestrabile anche su hardware limitato. AudioResNet, invece, è più indicato quando si cerca una maggiore capacità espressiva e si accetta un costo computazionale superiore. In questo progetto il confronto tra i due approcci mostra bene il classico compromesso tra efficienza e accuratezza: la rete più piccola è più snella, mentre quella più profonda tende a generalizzare meglio sulle configurazioni più difficili.

7 Conclusioni

In conclusione, il progetto conferma che il riconoscimento di comandi vocali è un problema naturale per le reti neurali convoluzionali quando l'audio viene trasformato in spettrogrammi Mel. La pipeline implementata è coerente e modulare: il dataset viene filtrato, il test set resta separato dal training, la rappresentazione tempo-frequenza viene costruita in modo uniforme e i modelli vengono valutati sia quantitativamente sia qualitativamente.

I risultati ottenuti mostrano che il sistema è già funzionale, ma lascia spazio a ulteriori miglioramenti. Le direzioni più interessanti riguardano l'augmentation audio, una migliore calibrazione degli iperparametri, l'uso di architetture più sofisticate o di tecniche di ensemble e un'analisi ancora più accurata dei casi errati. In sintesi, la parte decisiva del progetto non è soltanto la scelta del modello, ma il bilanciamento tra preprocessing, complessità della rete, qualità dei dati di input e capacità di interpretare correttamente le prestazioni ottenute.